División de Módulos de Trabajo — Sistema de Control de Ingreso de Aprendices (RFID)

Equipo: 2 integrantes **Criterio de división:** cada integrante es dueño de sus propias tablas de base de datos y sus propios archivos/controladores. Se define un único punto de contacto (la tabla `ingresos`) con un contrato claro para que ambos módulos puedan desarrollarse, probarse y desplegarse en paralelo sin pisarse el código.

Principio de la división

El sistema tiene dos grandes flujos:

1. **Flujo de identidad y marcaje (quién es quién, y quién entra/sale cuándo).**
2. **Flujo de seguimiento y justificación (qué se reporta y cómo se justifica una falta).**

Estos dos flujos comparten una sola tabla en común (`ingresos`), pero uno la **escribe** (Integrante A, vía RFID) y el otro solo la **lee** (Integrante B, para generar reportes). Esto permite trabajar en ramas Git independientes (`feature/nucleo-rfid` y `feature/reportes-excusas`) sin conflictos de merge, siempre que se respete el modelo de datos acordado desde el día 1.

Módulo 1 — Núcleo del Sistema: Usuarios, Fichas y Control RFID

Responsable: Integrante A

Historias de usuario que cubre (según la plantilla ágil)

ID	Rol	Historia
1	Admin	Crear/editar un aprendiz
2, 3, 4	Admin / Aprendiz / Instructor	Inicio de sesión con usuario y contraseña
5	Admin	Acceso a gestión de aprendices, instructores y asistencia

ID	Rol	Historia
8	Aprendiz	Registrar entrada/salida mediante código RFID
9	Admin	Registrar horario de cada ficha
10	Admin	Editar horario de ficha (entrada, salida, instructor)
14	Admin	Desactivar aprendiz que no continúa en formación
16	Sistema	Verificar si el aprendiz llegó en el horario correspondiente
17	Sistema	Eliminar automáticamente registros de quienes cumplieron el horario
20	Admin	Crear/editar un instructor

Tablas de base de datos que posee (lectura y escritura)

- `usuarios` (login, roles: admin / instructor / aprendiz)
- `aprendices`
- `instructores`
- `fichas`
- `horarios`
- `ingresos` *(dueño de escritura — ver contrato de datos abajo)*

Funcionalidades backend

- Autenticación y manejo de sesión/roles (login único reutilizado por los 3 roles).
- CRUD de aprendices e instructores.
- CRUD de fichas y horarios (incluye reasignación de instructor).
- Endpoint/lógica de marcaje RFID: recibe el código, identifica al aprendiz, registra hora de entrada o salida en `ingresos`.
- Job/proceso automático de fin de jornada: compara `ingresos` contra `horarios` y elimina los registros que cumplieron correctamente, dejando solo retardos, salidas anticipadas e inasistencias.

Funcionalidades frontend

- Pantalla de login.
- Panel admin: gestión de aprendices, instructores, fichas y horarios.
- Vista/simulación de lectura RFID (marcaje de entrada/salida).

Entregables propios de este módulo

- Modelo relacional de las tablas listadas arriba.
- Casos de uso: "Iniciar sesión", "Registrar/editar aprendiz", "Registrar/editar instructor", "Configurar horario de ficha", "Registrar entrada/salida RFID".
- Diagrama de actividades del proceso de marcaje y del proceso automático de fin de jornada.

Módulo 2 — Reportes, Excusas Médicas y Autoservicio del Aprendiz

Responsable: Integrante B

Historias de usuario que cubre

ID	Rol	Historia
6	Aprendiz	Consultar su historial de asistencia
7	Aprendiz	Anexar excusa médica
11	Admin	Generar reportes de inasistencia en PDF y Excel
12	Admin	Generar reportes por día, semana o rango personalizado
13	Instructor	Ver y validar la excusa médica del aprendiz
15	Aprendiz	Generar reporte personal de inasistencias y retardos

ID	Rol	Historia
18	Sistema	Las excusas aprobadas no cuentan como inasistencia
19	Sistema	Solo se reflejan inasistencias, salidas anticipadas y retardos reales

Tablas de base de datos que posee (lectura y escritura)

- `excusas_medicas` (aprendiz, ficha, fecha, archivo adjunto, estado: pendiente/aprobada/rechazada)
- `reportes` (si se decide persistir metadatos de reportes generados; opcional)
- `ingresos` *(solo lectura — nunca escribe aquí, ver contrato de datos abajo)*

Funcionalidades backend

- CRUD de excusas médicas (subir, listar, cambiar estado).
- Lógica que cruza `excusas_medicas` con `ingresos`: si hay excusa aprobada para una fecha, esa fecha no cuenta como inasistencia en ningún reporte.
- Generación de reportes agregados (por aprendiz, por ficha, por instructor) filtrando por día/semana/rango de fechas.
- Exportación de reportes a PDF y Excel.

Funcionalidades frontend

- Panel del aprendiz: historial personal de asistencia, retardos e inasistencias, y estado de sus excusas.
- Formulario para anexar excusa médica (con archivo adjunto).
- Panel del instructor: bandeja de excusas pendientes, aprobar/rechazar.
- Panel admin: generación y descarga de reportes (PDF/Excel) con filtros de fecha.

Entregables propios de este módulo

- Modelo relacional de `excusas_medicas` (y `reportes` si aplica).
- Casos de uso: "Anexar excusa médica", "Validar excusa médica", "Consultar historial personal", "Generar reporte de asistencia (PDF/Excel)".
- Diagrama de actividades del proceso de gestión de excusas y del proceso de generación de reportes.

Contrato de datos compartido (evita choques entre módulos)

Para que ambos trabajen en paralelo sin bloquear al otro, hay que fijar esto **antes de programar**, idealmente en la primera reunión de sprint:

1. **Estructura de la tabla `ingresos`**, acordada por ambos, por ejemplo: `id, id_aprendiz, id_ficha, fecha, hora_entrada, hora_salida, estado (a_tiempo / retardo / salida_anticipada / inasistencia)`.
    - Integrante A es el único que hace `INSERT`/`UPDATE`/`DELETE` sobre esta tabla.
    - Integrante B solo hace `SELECT` sobre esta tabla para armar reportes; nunca la modifica.
2. **Estructura de `aprendices`, `instructores` y `fichas`** (nombres de columnas e IDs) debe estar definida por Integrante A y comunicada a Integrante B, ya que `excusas_medicas` y los reportes referencian esos IDs por llave foránea.
3. **Regla de negocio de excusas** (HU 18): se recomienda implementarla como una función/vista que Integrante B expone (`obtener_inasistencias_reales(aprendiz_id, rango_fechas)`), para que el módulo de reportes sea el único responsable de "limpiar" inasistencias justificadas — así Integrante A no necesita saber nada de excusas.
4. **Autenticación:** Integrante A construye el login una sola vez (tabla `usuarios` con rol); Integrante B solo consume la sesión/rol ya autenticado para mostrar u ocultar sus propias pantallas (aprendiz/instructor/admin), sin tocar la lógica de login.

Recomendaciones de trabajo en equipo

- Repositorio Git con dos ramas de trabajo: `feature/nucleo-rfid` (Integrante A) y `feature/reportes-excusas` (Integrante B), integrando a `main`/`develop` mediante pull requests.
- El script de creación de base de datos (`schema.sql`) se versiona junto al repo y solo se modifica en conjunto cuando se toque una tabla compartida (`ingresos`, `aprendices`, `fichas`).
- Documentación SCRUM (backlog, sprints, daily) puede organizarse con estas mismas 2 columnas: "Núcleo/RFID" y "Reportes/Excusas", reflejando exactamente esta división de módulos.